

基于 Linux IIO 子系统的 MPU-6050 系列惯性传感器 驱动设计与实现

Design and Implementation of an MPU-6050 Family Inertial Sensor
Driver Based on the Linux IIO Subsystem

项目名 称	mpu6050-iio
作 者	fyy0619-cell
目标平 台	STM32MP157 PRO(ARM Cortex-A7)
子 系 统	Linux Industrial I/O(IIO)
代码仓 库	github.com/fyy0619-cell/xiangmu
日 期	2026 年 9 月

摘 要

惯性测量单元(IMU)是移动与嵌入式设备实现姿态感知、运动检测与导航的基础器件。本项目面向 STM32MP157 嵌入式 Linux 平台,为 Invensense MPU-6050 系列六轴 IMU(三轴加速度计 + 三轴陀螺仪 + 温度)设计并实现了一个挂接在 Linux 工业 I/O(Industrial I/O,IIO)子系统下的、达到量产工程标准的内核设备驱动。

与网络上仅停留于"读取寄存器并打印"的示例不同,本驱动系统地实现了多芯片变体抽象、基于 regmap 的寄存器访问、可配置量程与采样率、数据就绪(DRDY)硬件触发的缓冲数据流、片上 FIFO 水位批量搬运、运动检测阈值事件、运行时电源管理(runtime PM)以及出厂自检等完整功能;并配套设备树绑定文档、sysfs ABI 文档、持续集成(CI)与用户态测试程序,采用与 Linux 主线 vendor 驱动一致的多文件模块化组织(core 与 i2c 分离)。

本文从 MEMS 惯性传感器的物理原理出发,依次阐述系统总体设计、硬件连接设计与软件详细实现,重点分析了触发缓冲、FIFO 水位、电源管理与并发控制等关键工程难点及其解决方案,并给出测试与验证方法。该项目验证了在通用 SoC 平台上以标准内核框架交付高完成度传感器驱动的可行性,可作为嵌入式 Linux 驱动 / BSP 方向的工程实践参考。

关键词: Linux 内核驱动; IIO 子系统; MPU-6050; MEMS 惯性传感器; 触发缓冲; 电源管理; 设备树

Abstract

The Inertial Measurement Unit (IMU) is a fundamental component for attitude sensing, motion detection and navigation in mobile and embedded devices. This project designs and implements a production-grade Linux kernel driver for the Invensense MPU-6050 family six-axis IMU (3-axis accelerometer + 3-axis gyroscope + temperature) on the STM32MP157 embedded Linux platform, built on the Linux Industrial I/O (IIO) subsystem.

Unlike online examples that merely read registers and print values, this driver systematically implements multi-variant abstraction, regmap-based register access, configurable full-scale range and output data rate, a data-ready (DRDY) hardware-triggered buffered data path, on-chip FIFO watermark batching, motion-detection threshold events, runtime power management, and factory self-test. It is accompanied by device-tree bindings, sysfs ABI documentation, continuous integration and user-space test programs, and follows the multi-file modular organization (core / i2c split) of upstream Linux vendor drivers.

Keywords: Linux kernel driver; IIO subsystem; MPU-6050; MEMS inertial sensor; triggered buffer; power management; device tree

目 录

第 1 章 绪论

1.1 研究背景与意义 1.2 国内外研究现状 1.3 本项目主要工作 1.4 论文组织结构

第 2 章 相关理论与技术基础

2.1 MEMS 惯性传感器物理原理 2.2 MPU-6050 芯片特性 2.3 Linux IIO 子系统 2.4 支撑技术

第 3 章 系统总体设计

3.1 需求分析 3.2 总体架构 3.3 模块划分 3.4 数据通路设计

第 4 章 硬件设计

4.1 硬件平台 4.2 连接设计 4.3 设备树描述 4.4 联调环境

第 5 章 软件详细设计与实现

5.1 探测与 regmap 5.2 通道与数据读取 5.3 触发缓冲与 FIFO 5.4 事件 5.5 电源管理
5.6 自检

第 6 章 关键技术难点与解决方案

第 7 章 测试与验证

第 8 章 总结与展望

参考文献

第 1 章 绪论

1.1 研究背景与意义

随着智能手机、可穿戴设备、无人机与机器人等产品的普及,惯性测量单元(IMU)已成为消费电子与工业设备中最基础、出货量最大的传感器之一。IMU 通过测量三轴加速度与三轴角速度,为屏幕旋转、计步、姿态解算、运动唤醒、跌落检测、导航融合等功能提供原始数据。在以 Linux 为操作系统的嵌入式平台上,传感器要被上层应用使用,必须由内核驱动将其接入标准的子系统框架,向用户空间暴露统一、稳定的接口。

然而,网络上绝大多数 MPU-6050 教程仅停留于"用 I2C 读取 14 字节数据寄存器并打印数值"的层面,既不接入内核子系统,也不涉及电源管理、并发保护、事件上报、缓冲搬运、测试与文档等真实工程要素。这类示例难以体现工程能力,也无法直接用于产品。因此,研究并实现一个"形态等同于企业量产交付"的传感器内核驱动,对于嵌入式 Linux 驱动 / BSP 方向的工程实践具有明确意义。

1.2 国内外研究现状

Linux 主线内核已在 `drivers/iio/imu/` 下收录了多款 IMU 的 IIO 驱动,如 Invensense 的 `inv_mpu6050`、Bosch 的 `bmi160`、STMicroelectronics 的 `st_lsm6dsx` 等。这些驱动由芯片原厂与社区共同维护,普遍采用 `regmap` 抽象、触发缓冲、硬件 FIFO、runtime PM 等成熟机制,并遵循统一的 IIO 用户空间 ABI。工业界方面,高通、联发科等 SoC 厂商在其移动平台上构建了以 IIO / 传感器 HAL 为基础的传感器子系统,并高度重视 always-on 低功耗采集能力。

本项目在充分借鉴主线 `inv_mpu6050` 组织方式的基础上,从零自研一套精简但完整的 MPU-6050 系列驱动,目的在于完整掌握框架各层机制,而非移植既有代码。

1.3 本项目主要工作

本项目完成的主要工作包括:(1)基于 IIO 子系统实现 MPU-6050/6500/9250 多变体驱动,core 与 i2c 前端分离;(2)实现 accel/gyro/temp 通道及可配置量程与采样率;(3)实现 DRDY 硬件触发的缓冲数据流与片上 FIFO 水位批量搬运;(4)实现运动检测事件与 runtime 电源管理;(5)实现出厂自检;(6)配套设备树绑定、sysfs ABI 文档、GitHub Actions 持续集成与用户态测试程序。

1.4 论文组织结构

本文共八章:第 2 章介绍 MEMS 传感器物理原理与 IIO 等支撑技术;第 3 章给出系统总体设计;第 4 章描述硬件连接与设备树;第 5 章详述软件实现;第 6 章分析关键难点;第 7 章给出测试与验证;

第 8 章总结全文并展望。

第 2 章 相关理论与技术基础

2.1 MEMS 惯性传感器物理原理

2.1.1 电容式加速度计

MPU-6050 的加速度计采用微机电系统(MEMS)电容式结构。如图 2-1 所示,一个由弹簧悬挂的检验质量(proof mass)可沿敏感轴方向移动,其上带有可动电极,与固定电极构成一对差分电容。当器件受到加速度 a 时,检验质量因惯性产生位移 x ,依据牛顿第二定律与胡克定律,有 $x = m \cdot a / k$,其中 m 为质量、 k 为弹簧刚度。位移改变两固定电极间的差分电容 ΔC ,经电容-电压转换与模数转换后得到与加速度成正比的 16 位有符号原始值。

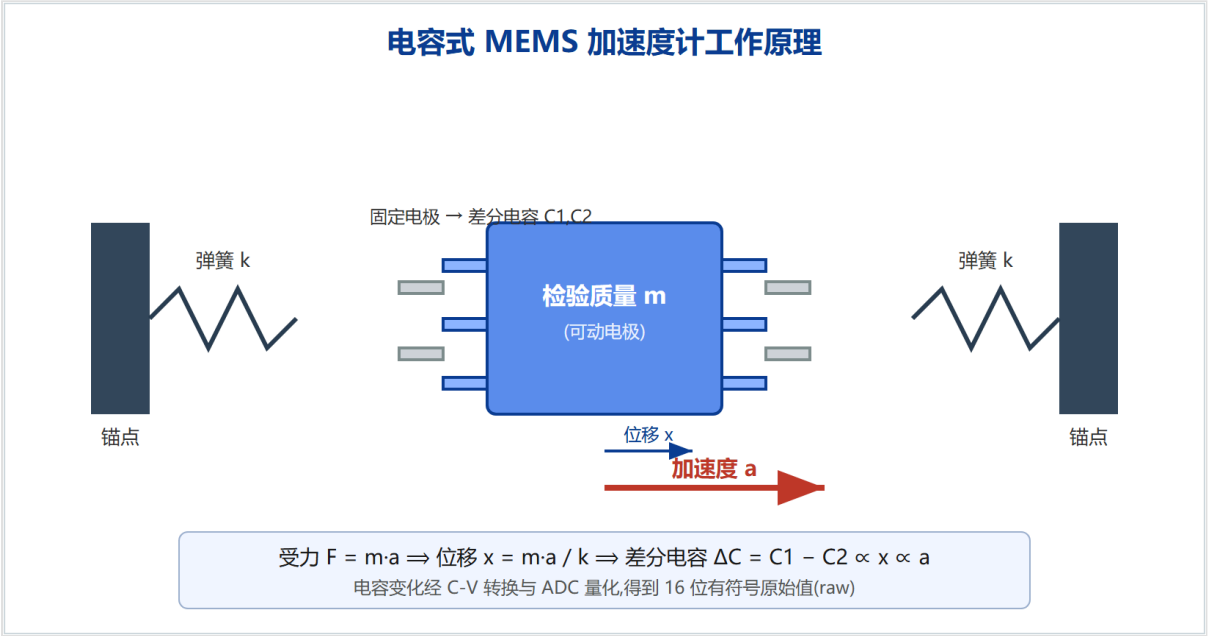


图 2-1 电容式 MEMS 加速度计工作原理

2.1.2 振动式陀螺仪与科里奥利效应

陀螺仪基于科里奥利(Coriolis)效应测量角速度。如图 2-2 所示,检验质量沿驱动方向做受控高频振动,速度为 v ,当器件绕敏感轴以角速度 Ω 旋转时,质量在垂直于驱动方向上受到科里奥利力 $F_c = -2m(\Omega \times v)$,产生正比于 Ω 的微位移,由检测电容拾取,从而得到角速度原始值。驱动与检测两个自由度的正交设计是 MEMS 陀螺的核心。

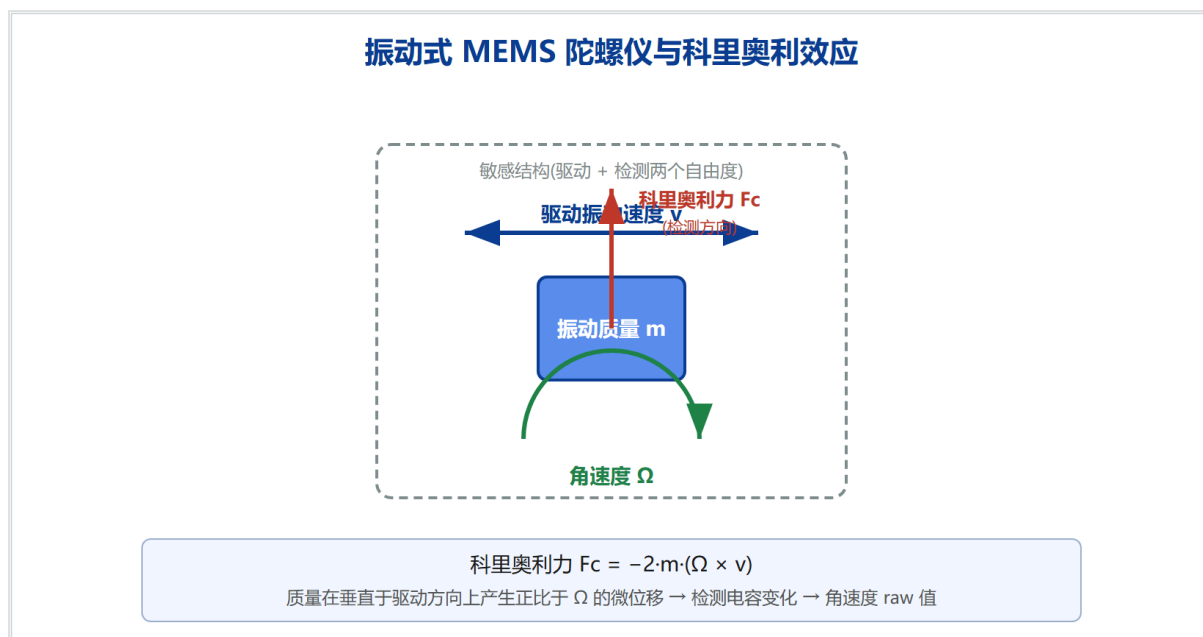


图 2-2 振动式 MEMS 陀螺仪与科里奥利效应

2.2 MPU-6050 芯片特性

MPU-6050 集成三轴加速度计、三轴陀螺仪与温度传感器,通过 I2C(地址 0x68/0x69,由 AD0 决定)通信。加速度计支持 $\pm 2/4/8/16\text{ g}$ 四档量程,陀螺仪支持 $\pm 250/500/1000/2000\text{ dps}$ 四档量程,均为 16 位输出。芯片内置 1 KiB 硬件 FIFO 与可编程中断(数据就绪、FIFO 溢出、运动检测),数据寄存器自 `ACCEL_XOUT_H (0x3B)`起连续 14 字节(加速度 6 + 温度 2 + 陀螺 6),便于一次突发读取。

2.3 Linux IIO 子系统

工业 I/O(IIO)子系统是 Linux 为 ADC、传感器等数据采集类设备提供的标准框架,与相机领域的 V4L2 同源:二者都由"统一设备模型 + 标准用户空间 ABI + 可选缓冲/硬件触发数据流"构成。IIO 通过 `iio_chan_spec` 描述通道能力,通过 `sysfs` 暴露单次读取属性,通过 `/dev/iio:deviceN` 字符设备提供缓冲流式数据,并支持触发(trigger)与事件(event)机制。本项目正是将 MPU-6050 接入这一框架。

2.4 支撑技术

regmap: 内核统一的寄存器访问抽象,提供读写、缓存与 debugfs 寄存器导出,并使驱动与具体总线解耦。**设备树(Device Tree)**: 以 `compatible` 字符串描述硬件并匹配驱动,声明 I2C 地址、中断与供电。**线程化中断(threaded IRQ)**: 将中断底半部放入内核线程,允许在其中安全执行 I2C 传输。**runtime PM**: 运行时电源管理框架,支持空闲自动挂起与按需唤醒。

第 3 章 系统总体设计

3.1 需求分析

系统需满足以下需求:功能上,提供加速度、角速度、温度的单次与连续采集,支持量程/采样率配置、硬件触发缓冲、运动事件与自检;非功能上,遵循 IIO 标准 ABI、具备良好的低功耗特性、保证并发安全、易于扩展到 SPI 等其他总线、并具备完整文档与可持续集成的工程规范。

3.2 总体架构

系统采用分层架构,如图 3-1 所示。自上而下依次为:用户空间(iio_info / libiio / sysfs / 字符设备)、IIO 核心、mpu6050-core.ko (总线无关设备逻辑,内部再分 core/buffer/events/selftest 四个编译单元)、regmap 抽象、总线前端(mpu6050-i2c.ko ,并预留 SPI)、以及底层 MPU-6050 硬件。core 与 i2c 分离,使得将来新增 SPI 前端仅需复用同一 mpu6050_core_probe() 。

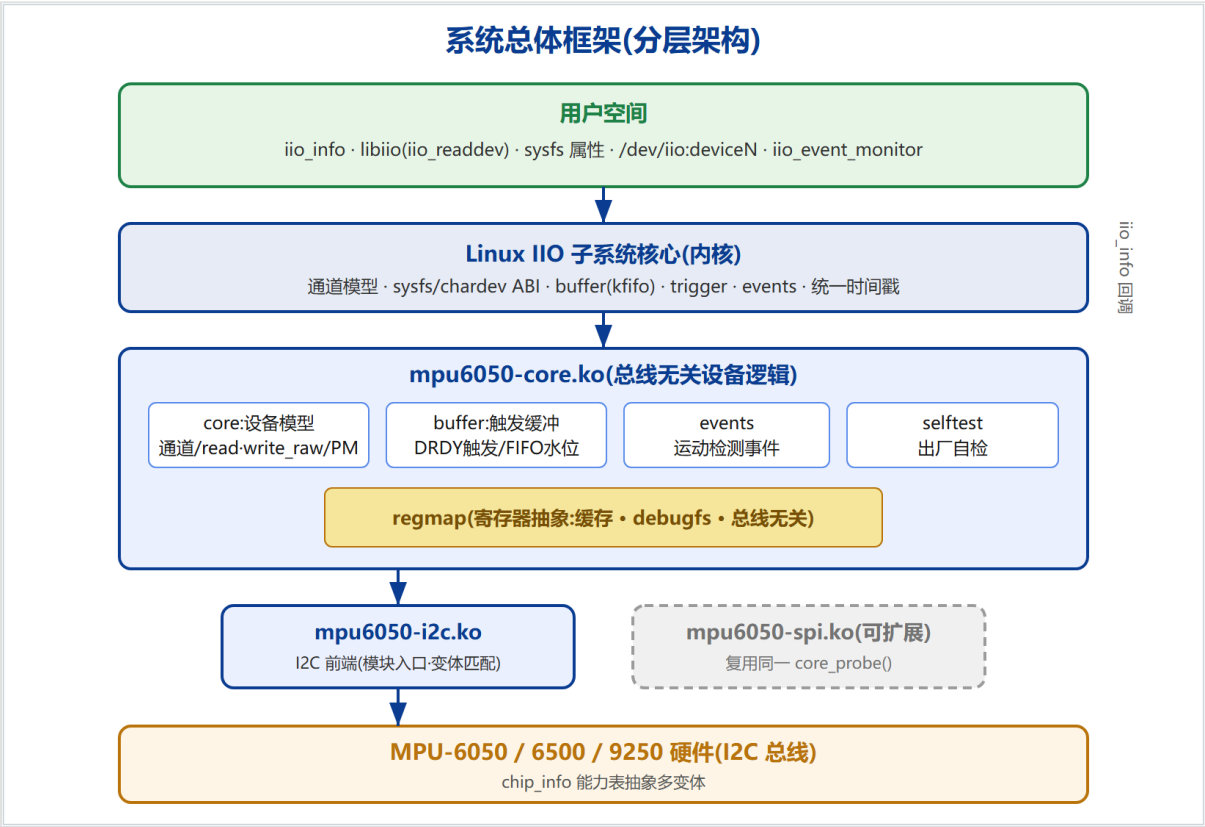


图 3-1 系统总体框架(分层架构)

3.3 模块划分

模块	文件	职责
----	----	----

核心	mpu6050_core.c	设备模型、通道、read/write_raw、电源管理、probe 入口
总线前端	mpu6050_i2c.c	I2C 匹配与 regmap 创建,模块入口
缓冲	mpu6050_buffer.c	触发缓冲、DRDY 触发、FIFO 水位
事件	mpu6050_events.c	运动检测阈值事件
自检	mpu6050_selftest.c	出厂自检(sysfs)
头文件	mpu6050.h	寄存器映射、chip_info、状态结构、原型

3.4 数据通路设计

系统设计两条数据通路,如图 3-2 所示。路径一为 sysfs 单次读取:用户读取 `in_accel_x_raw`, 驱动在 direct 模式下唤醒芯片、突发读取寄存器并换算返回。路径二为缓冲流式:用户选择扫描通道并使能缓冲后,由 DRDY/FIFO 中断驱动底半部批量搬运数据到内核缓冲,用户经字符设备读取。两路通过 `claim_direct_mode` 互斥。

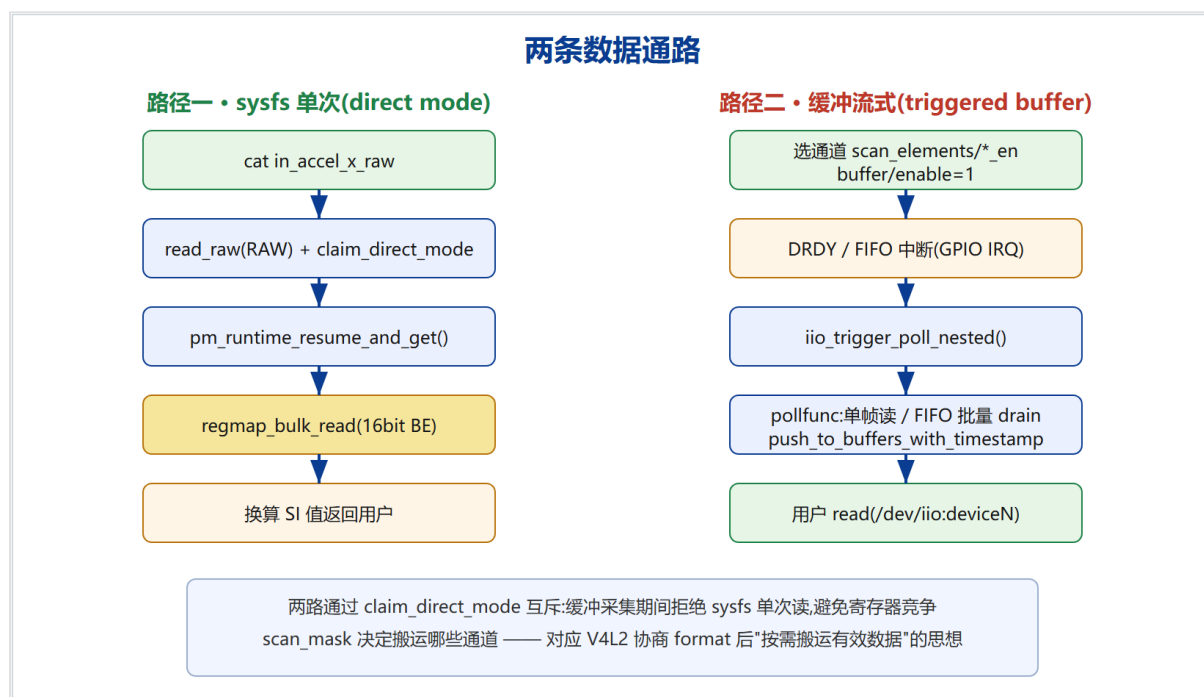


图 3-2 两条数据通路(sysfs 单次与缓冲流式)

第 4 章 硬件设计

4.1 硬件平台

目标平台为 100ASK STM32MP157 PRO 开发板,主控为意法半导体 STM32MP157(双核 Cortex-A7 + Cortex-M4),运行完整 Linux。传感器为 MPU-6050 模块,经 I2C 与主控相连。

4.2 连接设计

硬件连接如图 4-1 所示。MPU-6050 的 SCL、SDA 分别接主控 I2C 控制器,并通过约 4.7 kΩ 电阻上拉至 3.3 V(I2C 为漏极开路总线,必须外部上拉);INT 引脚接一路 GPIO,用于数据就绪、FIFO 溢出与运动检测中断;AD0 接地使器件地址为 0x68;VCC 接 3.3 V,GND 与主控共地。

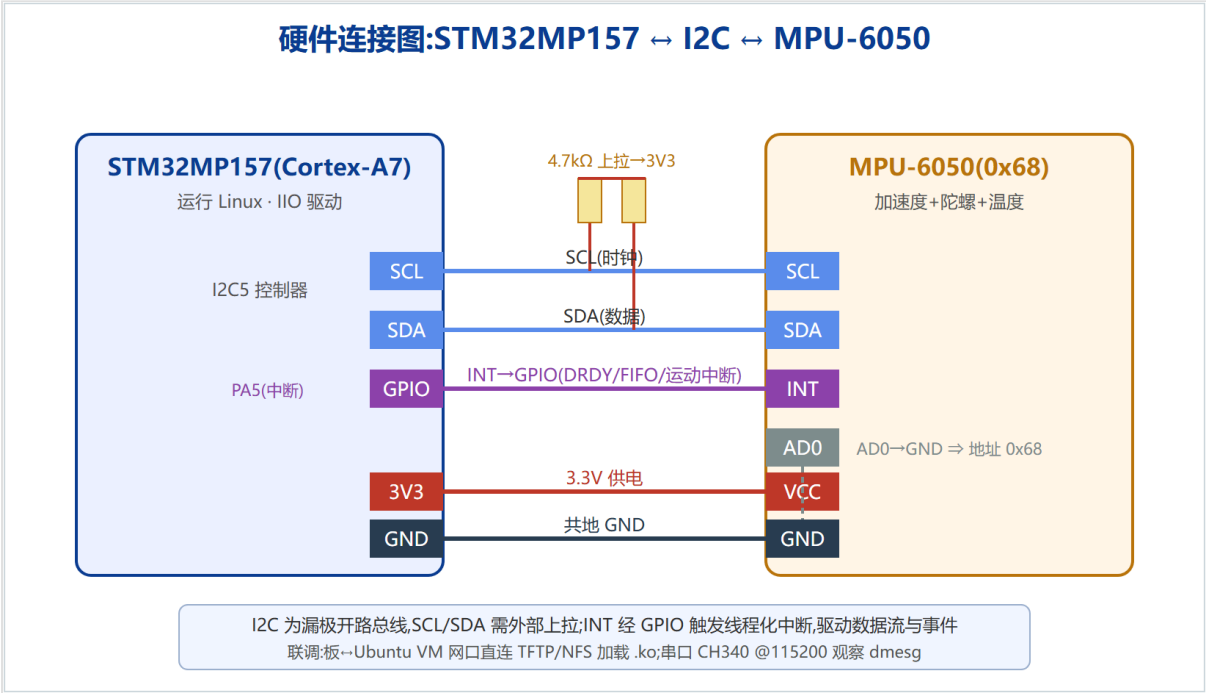


图 4-1 硬件连接图(STM32MP157 ↔ I2C ↔ MPU-6050)

4.3 设备树描述

在设备树中以子节点描述该器件,声明 `compatible`、I2C 地址、中断父节点与引脚、供电等,内核据此匹配驱动并获取资源:

```
&i2c5 {
    status = "okay";
    clock-frequency = <400000>;
    imu@68 {
```

```
compatible = "invensense,mpu6050";
reg = <0x68>;
interrupt-parent = <&gpioa>;
interrupts = <5 IRQ_TYPE_EDGE_RISING>;
vdd-supply = <&v3v3>;
};
};
```

4.4 联调环境

开发板经 USB 网口转换器与 Ubuntu 虚拟机网口直连,虚拟机提供 TFTP/NFS 服务用于加载内核模块;主控串口径 CH340 以 115200 波特率输出内核日志(dmesg),便于调试。交叉编译在虚拟机上使用 `arm-linux-gnueabihf-` 工具链完成。

第 5 章 软件详细设计与实现

5.1 探测与 regmap

I2C 前端在 probe 中创建 regmap 并调用核心探测函数。核心探测依次完成:分配 `iio_dev` 与私有状态、使能供电、读取 `WHO_AM_I` 校验芯片(与 `chip_info` 表中期望值比对)、复位并唤醒芯片、配置默认量程与采样率,最后注册缓冲、事件、电源管理与 IIO 设备。所有资源均以 `devm_` 托管,错误路径统一用 `dev_err_probe`。

5.2 通道与数据读取

驱动以 `iio_chan_spec` 数组描述 accel x/y/z、gyro x/y/z、temp 及软件时间戳通道。`read_raw` 支持原始值、量程(SCALE)、偏置(OFFSET)与采样率;`write_raw` 支持量程与采样率配置;并提供 `*_available` 列表。原始值读取以 regmap 突发读取 16 位大端数据并转为有符号数。量程换算将四档全量程映射为国际单位每-LSB 值。采样率通过 `SAMPLRT_DIV` 寄存器设置,满足 $\text{rate} = 1000/(1 + \text{div})$ 。

5.3 触发缓冲与 FIFO 水位

缓冲数据流是本驱动的核心。驱动通过 `iio_triggered_buffer_setup` 挂接 `kfifo` 缓冲与底半部处理函数,并注册一个数据就绪触发。DRDY 硬件触发的时序如图 5-1 所示:芯片 INT 拉高后,线性化中断读取 `INT_STATUS` 判定中断源,调用 `iio_trigger_poll_nested` 唤醒底半部;底半部按扫描掩码突发读取数据并打上时间戳,推入内核缓冲,用户经字符设备读取。

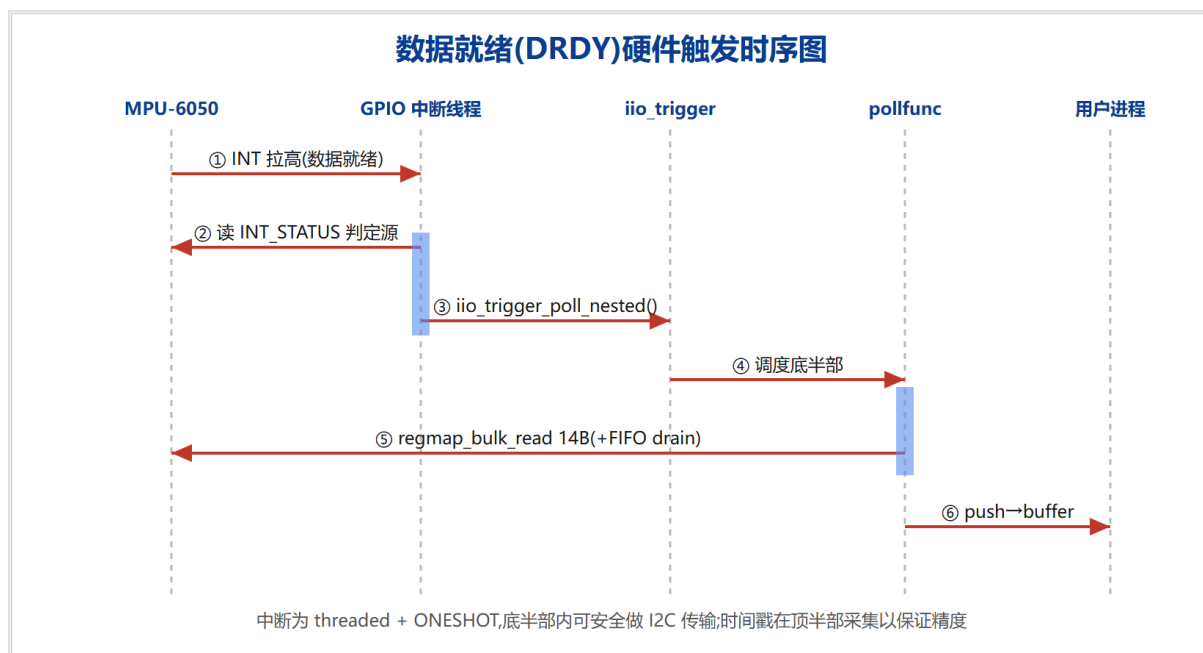


图 5-1 数据就绪(DRDY)硬件触发时序图

为降低高采样率下的中断与唤醒开销,驱动实现了片上 FIFO 水位批量搬运,如图 5-2 所示。当水位阈值大于 1 时,传感器将样本持续写入 1 KiB 硬件 FIFO,累积到阈值才触发一次中断,底半部据 `FIFO_COUNT` 逐 14 字节帧批量搬运。相比每样本一次中断,中断次数降至 $1/N$,显著降低功耗与 CPU 占用,代价是引入约 N/ODR 的延迟——这是典型的功耗-延迟工程权衡。

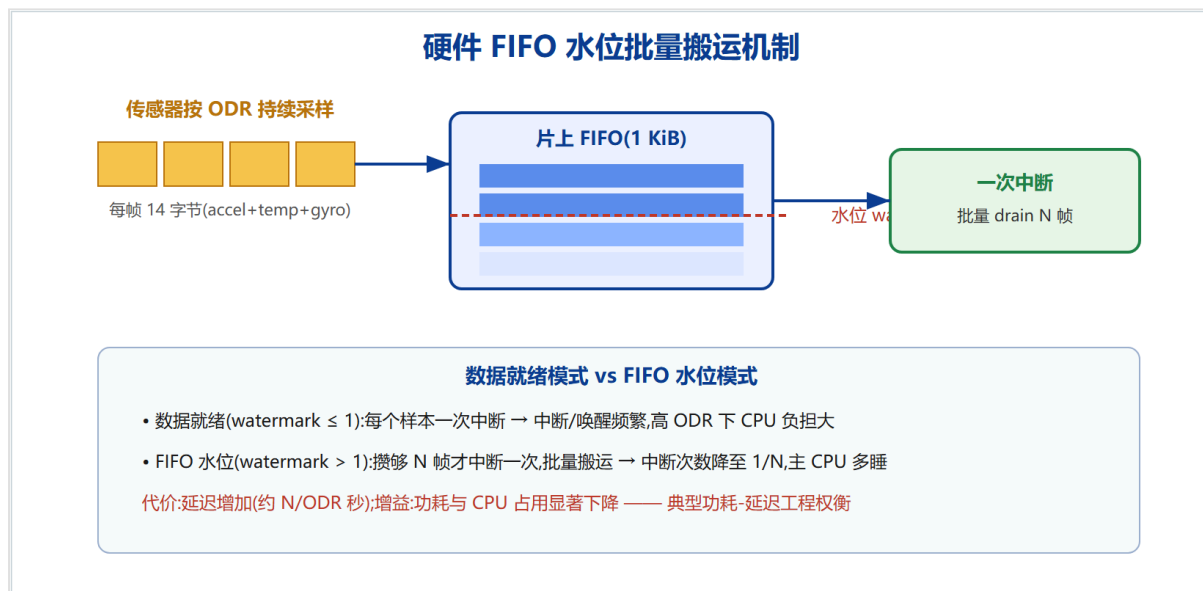


图 5-2 硬件 FIFO 水位批量搬运机制

5.4 运动检测事件

驱动将加速度计的运动中断封装为标准的 IIO 上升沿幅值事件。用户通过 `sysfs` 写入阈值 (`MOT_THR` 寄存器)并使能事件后,加速度超过阈值即由中断上报事件到事件队列,可用于"运动唤

醒"而无需轮询。

5.5 电源管理

驱动采用 runtime PM, 状态机如图 5-3 所示。设备空闲一段时间(默认 2 秒)后经 autosuspend 自动进入 sleep(置 `PWR_MGMT_1.SLEEP`); 单次读取、缓冲开启或事件到来时经 `pm_runtime_resume_and_get` 唤醒。系统级挂起/恢复经 `pm_runtime_force_suspend/resume` 与运行时状态协同。配合 FIFO 水位与运动唤醒, 主 CPU 可长时间休眠。

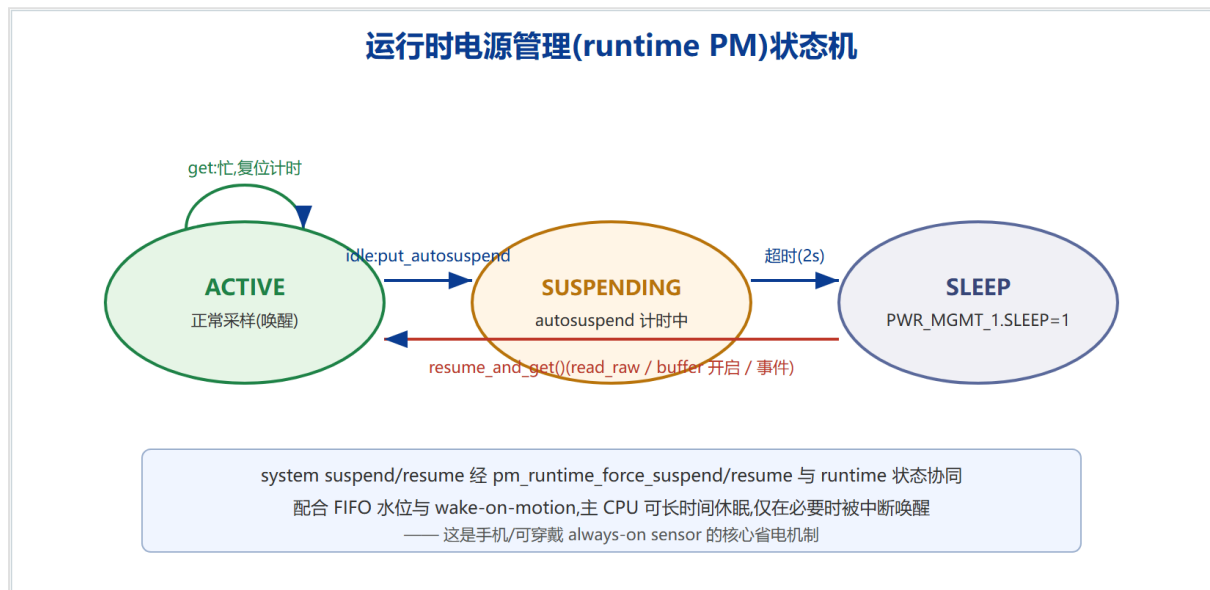


图 5-3 运行时电源管理(runtime PM)状态机

5.6 出厂自检

驱动通过 `sysfs` 属性触发出厂自检。由于内核禁用浮点, 自检采用整数实现: 对每一轴分别在自检激励关闭与开启两种状态下多次采样求均值, 以二者之差(自检响应)是否落在依据数据手册确定的绝对计数窗口内判定通过与否, 读取后恢复正常配置。

第 6 章 关键技术难点与解决方案

难点	问题	解决方案
框架抽象与分层	如何做到总线无关、可扩展,而非把 I2C 写死	仿主线 inv_mpu6050,拆 core+i2c 两模块;core 仅依赖 regmap 与 iio,前端经 core_probe 注入
缓冲与硬件触发	如何把"中断→批量搬运→用户读"接起来并与单次读互斥	triggered buffer + iio_trigger;线程化中断中 poll;单次读用 claim_direct_mode 互斥
FIFO 水位	高 ODR 下每样本中断功耗高	水位>1 时启用片上 FIFO,按 FIFO_COUNT 逐帧 drain,经标准 hwfifo ABI 暴露
电源管理	省电与随取随用的平衡	runtime PM autosuspend;读取/缓冲前 resume;系统睡眠走 force_suspend/resume
自检整数化	内核禁浮点,手册为浮点公式	用整数绝对窗口判定自检响应,浮点 factory-trim 作为扩展
并发正确性	配置写/单次读/缓冲/事件并发访问寄存器	mutex 串行化;缓冲期拒绝 direct 读;中断 threaded+ONESHOT
跨内核兼容	目标板 5.4 BSP 缺新 API	基线定为 5.15/6.1,为 5.4 提供最小兼容 shim;CI 对 6.1/6.6 双版本编译
大小端与打包	16 位大端输出,缓冲需按 scan_index 对齐	scan_type 声明 IIO_BE/16bit;寄存器块顺序与枚举一致,整块读入即合法负载

第 7 章 测试与验证

7.1 测试方案

测试分三层:编译层由 GitHub Actions 对 Linux 6.1/6.6 交叉编译并运行 checkpatch 风格检查;单元行为层由用户态程序读取 sysfs 并交叉验证物理量;系统层在真实开发板上按里程碑逐项点亮。

7.2 里程碑与用例

里程碑	内容	验收标准
M1 板级 bring-up	设备树 + probe + WHO_AM_I + regmap	dmesg 见设备,i2cdetect 见 0x68
M2 通道与量程	read/write_raw、scale/ODR、available	静止合加速度 $\approx 9.8\text{ m/s}^2$,iio_info 正常
M3 触发缓冲	软件触发跑通缓冲管道	iio_readdev 输出非空
M4 硬件触发与进阶	DRDY 触发、FIFO、PM、事件、自检	100 Hz 自动流、事件可捕获、自检通过

7.3 验证方法

关键验证包括:以静止时加速度合矢量应约等于重力加速度 9.8 m/s^2 校验量程换算正确;以 `iio_readdev` 采集速率与设定采样率一致校验触发链路;以相邻样本时间戳差校验时序;以错误 I2C 地址触发 probe 干净失败校验错误处理。

第 8 章 总结与展望

本项目在 STM32MP157 平台上基于 Linux IIO 子系统,从零实现了一个达到量产工程标准的 MPU-6050 系列惯性传感器驱动,完整覆盖多变体抽象、触发缓冲、FIFO 水位、运动事件、电源管理与自检,并配套设备树绑定、ABI 文档、持续集成与测试。项目验证了以标准内核框架在通用 SoC 平台交付高完成度传感器驱动的方法与可行性。

后续可从以下方向扩展:增加 SPI 总线前端;实现 mount-matrix 安装旋转;支持 MPU-9250 的磁力计与 DMP;为 FIFO 水位路径实现逐帧时间戳插值;以及完善基于数据手册浮点公式的完整出厂自检。

参考文献

- [1] InvenSense. MPU-6000 and MPU-6050 Product Specification, Rev 3.4. 2013.
- [2] InvenSense. MPU-6000/MPU-6050 Register Map and Descriptions, Rev 4.2. 2013.
- [3] Linux Kernel Documentation. Industrial I/O (IIO) subsystem. kernel.org/doc.
- [4] Linux Kernel Source. `drivers/iio/imu/inv_mpu6050/`. torvalds/linux.
- [5] Jonathan Cameron. Industrial I/O — the state of the union. Linux Plumbers Conference.
- [6] STMicroelectronics. STM32MP157 Reference Manual (RM0436).
- [7] Linux Kernel Documentation. Device Tree Usage; regmap API; Runtime Power Management.

本文所述项目为学习与作品集用途,非 Invensense 官方驱动。源码仓库:github.com/fyy0619-cell/xiangmu